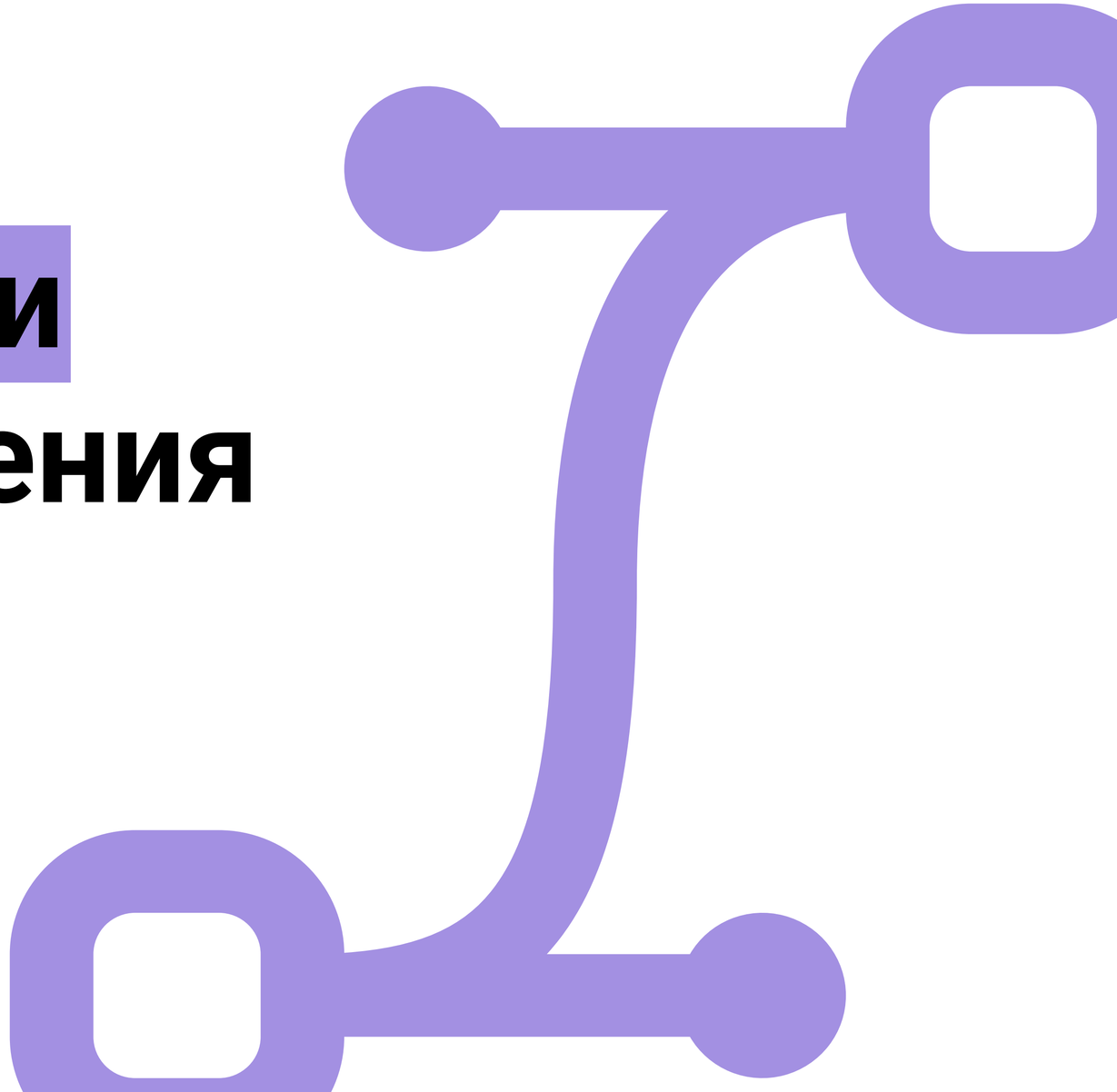
2026

Платформы и среды выполнения ЯП

NEXIGN x ИТМО

Раздел 02

Ландшафт памяти и модели исполнения



Управление памятью: две беды

Объект освобождён слишком рано

(use-after-free)

Объект больше не нужен, но продолжает удерживаться

(memory leak)

**Кто и на каком основании решает,
что объект больше не нужен?**



Что делает аллокатор (на самом деле)

Учебная модель: first-fit, best-fit, next-fit...

В реальности:

- Аллокатор может получать память у ОС разными способами.
- `free` не гарантирует немедленного уменьшения RSS.
- Виртуальный размер, RSS и объём живых объектов — разные величины.

Модели управления памятью

- Ручное управление — C.
- RAII и владение через типы — C++.
- Подсчёт ссылок — `shared_ptr`, CPython, Swift ARC.
- Tracing GC — Java, C#, JavaScript, Go.
- Владение и заимствование — Rust.

Замечание: управление памятью — это спектр, модели могут сочетаться.

Ручное управление в C/C++

- `malloc`: выделяет хранилище, не конструирует C++-объект.
- `free`: освобождает совместимо выделенное хранилище.
- `new`: выделение и инициализация объекта.
- `delete`: завершение времени жизни объекта и освобождение хранилища через соответствующий механизм.

Примечание: стандарт C++ не требует, чтобы new внутри вызывал `malloc`.

Проблемы: `утечка, use-after-free, double free, выход за границы...`

Хранилище, объект, время жизни

`хранилище выделено → объект создан → объект уничтожен → хранилище освобождено`

Владение

Владение — ответственность за жизненный цикл; оно не тождественно наличию адреса.

RAII (Resource Acquisition Is Initialization)

- Ресурс принадлежит объекту-обёртке.
- Деструктор освобождает ресурс.
- Очистка происходит и при выходе из области видимости через исключение.
- Подход применим к памяти, файлам, блокировкам и другим ресурсам.

Выделение динамической памяти выполняется при создании объекта, а не где-то еще в коде. Основной инструмент RAII для управления динамически выделяемой памятью в C++: умные указатели.

Владение в Rust

Владение — это основной принцип управления памятью в Rust, где каждое значение имеет переменную, называемую его *владельцем*.

Одновременно может существовать только один владелец для каждого значения. Когда владелец выходит из области видимости, значение и память, которую оно занимает, автоматически удаляются.

Владение в C++

- `unique_ptr`: единоличное владение; передача через move; автоматическое освобождение.
- `shared_ptr`: совместное владение. Несколько сильных владельцев; объект уничтожается при исчезновении последнего; дополнительные накладные расходы; не делает сам объект потокобезопасным.
- `weak_ptr`: наблюдение без владения. Не удерживает объект сильной ссылкой; `lock()` проверяет время жизни.

Подсчёт ссылок

Концептуальная схема:

Сильная ссылка появилась → счётчик увеличился; сильная ссылка исчезла → счётчик уменьшился

Счётчик стал нулём → объект может быть уничтожен.

Проблемы:

- циклические ссылки;
- затраты на учёт ссылок;
- синхронизация в некоторых многопоточных сценариях;
- каскадное уничтожение объектов;
- освобождение объекта не гарантирует немедленного возврата памяти ОС.

CPython и Swift ARC

CPython: подсчёт ссылок и дополнительный сборщик циклических ссылок; детали зависят от версии.

Swift ARC: автоматическое управление сильными ссылками; weak и unowned выражают разные предположения о времени жизни.

Инструменты анализа памяти

Инструмент	Для чего подходит
AddressSanitizer	Выходы за границы, use-after-free и ряд других ошибок доступа
LeakSanitizer	Обнаружение ряда утечек в поддерживаемых конфигурациях
Valgrind Memcheck	Ошибки доступа, неинициализированные значения, утечки
Профилировщик памяти	Источники и динамика потребления памяти
RSS и системные метрики	Резидентная память процесса, но не прямой счётчик живых объектов

Живые объекты, heap и RSS

Объект может стать недостижимым, но ещё не быть собран; runtime может сохранить зарезервированные области, а RSS не обязан немедленно уменьшаться. Один график RSS не доказывает утечку; сопоставлять его нужно с heap-профилем и метриками runtime…

Tracing GC

Управляемая среда: runtime управляет памятью объектов автоматически.

Примеры: Java/JVM, C#/.NET, JavaScript/V8 и Go.

Примечание: внешние ресурсы по-прежнему требуют явного освобождения. Сборка не обещает уничтожение объекта сразу после последнего использования.

Tracing GC


Когда ваш код генерит больше
мусора, чем городская служба переработки
imgflip.com

Достижимость

Достижимость (reachability) в контексте Garbage Collection (GC) — это свойство объекта, определяющее, может ли программа каким-либо образом обратиться к нему в будущем. Если объект достижим, сборщик мусора обязан сохранить его в памяти; если недостижим — он считается мусором и подлежит удалению.

Mark-and-Sweep

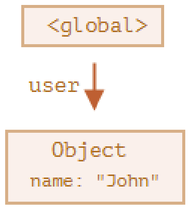
Простыми словами:

1. Обход от корней и маркировка достижимых объектов.
2. Проход по управляемой области памяти.
3. Освобождение непомеченных объектов.

Mark-and-Sweep

Вот самый простой пример:

```
// в user находится ссылка на объект
let user = {
  name: "John"
};
```



Если перезаписать значение `user`, то ссылка потеряется:

```
user = null;
```



https://learn.javascript.ru/garbage-collection